



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

3. Sortieren



Sortieren

- Grundlegendes Problem in der Informatik
- Aufgabe:
 - **Ordnen** von Dateien mit Datensätzen, die **Schlüssel** enthalten
 - Umordnen der Datensätze, so dass klar definierte Ordnung der Schlüssel (numerisch/alphabetisch) besteht
 - Modell / Vereinfachung: nur Betrachtung der Schlüssel, z.B. Feld von int-Werten
- Warum?
 - binäre Suche vs. sequentielle Suche!
- Wann lohnt sortieren nicht?



Ordnungsrelation

- Eine Relation, die reflexiv, antisymmetrisch und transitiv ist, heißt **Ordnungsrelation**.
 - Zwei Elemente aus A heißen **vergleichbar** bezüglich R, wenn $x \leq_R y$ oder $y \leq_R x$ gilt.
 - Eine Ordnungsrelation R in A heißt **total**, wenn jedes Element aus A mit jedem vergleichbar ist.
 - Andernfalls heißt R **partiell**.
 - Bei einer totalen Ordnungsrelation R kann man sich die Elemente aus A in einer bestimmten Reihenfolge **linear angeordnet** denken.
 - Also kann man sie sortieren!



Sortieren: Grundbegriffe

- Verfahren
 - **intern**: in Hauptspeicherstrukturen (Felder, Listen)
 - **extern**: Datensätze auf externen Medien (Festplatte, Magnetband)
- Stabilität
 - Ein Sortierverfahren heißt **stabil**, wenn es die relative Reihenfolge gleicher Schlüssel in der Datei beibehält.

<u>Name</u>	<u>Alter</u>		<u>Name</u>	<u>Alter</u>
Endig, Martin	30		Höpfner, Hagen	24
Geist, Ingolf	28		Geist, Ingolf	28
Höpfner, Hagen	24		Schallehn, Eike	28
Schallehn, Eike	28		Endig, Martin	30

Sortieren durch Einfügen



- Starte einen Stapel mit der ersten Karte.
- Wiederhole
 - Nehme die nächste Karte und füge sie an der richtigen Stelle in den neuen Stapel ein, bis alle Karten einsortiert sind.
- Algorithmisch: Aktuelle Karte „merken“, nach links die richtige Einfügeposition suchen, alle größeren Elemente auf dem Weg eins nach rechts schieben



Sortieren durch Einfügen

algorithm InsertionSort (F)

Eingabe/Ausgabe: zu sortierende Folge F der Länge n

```
for i := 2 to n do
  m := F[i]; /* zu merkendes Element */
  j := i;
  while j > 1 do
    if F[j-1] > m then
      /* verschiebe F[j-1] um eine Position nach rechts */
      F[j] := F[j-1];
      j := j - 1;
    else
      Verlasse innere Schleife
  fi
od;
F[j] := m /* j zeigt auf Einfügeposition */
od;
```



Sortieren durch Einfügen

algorithm InsertionSort (F)

Eingabe/Ausgabe: zu sortierende Folge F der Länge n

for i := 2 to n do

 m := F[i]; /* zu merkendes Element */

 j := i;

 while j > 1 do

 if F[j-1] > m then

 /* verschiebe F[j-1] um eine Position nach rechts */

 F[j] := F[j-1];

 j := j - 1;

 else

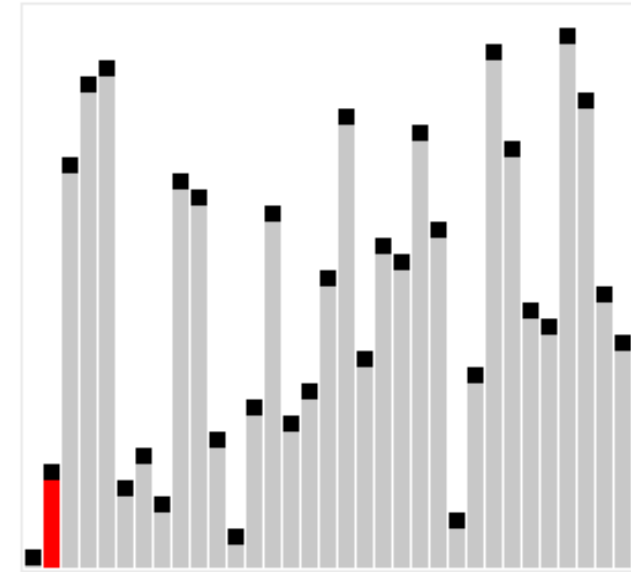
 Verlasse innere Schleife

 fi

 od;

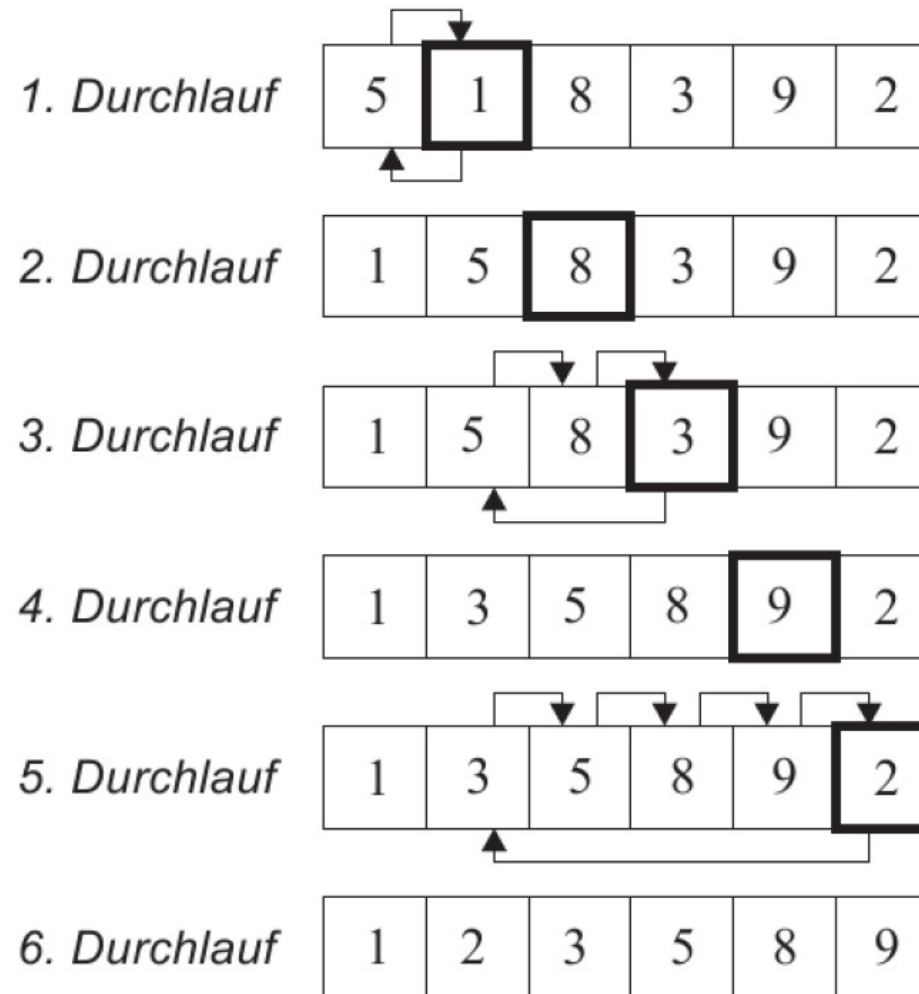
 F[j] := m /* j zeigt auf Einfügeposition */

od;



https://commons.wikimedia.org/wiki/File:Sorting_insertion_sort_anim.gif#/media/File:Sorting_insertion_sort_anim.gif

Beispiel: Sortieren durch Einfügen





InsertionSort in Java

```
static void insertionSort(int[] array) {  
    for (int i = 1; i < array.length; i++) {  
        int marker = i;  
        int temp = array[i];  
        // für alle Elemente links vom Marker-Feld  
        while (marker > 0 && array[marker - 1] > temp) {  
            // verschiebe alle größeren Element nach hinten  
            array[marker] = array[marker - 1];  
            marker--;  
        }  
        // setze temp auf das freie Feld  
        array[marker] = temp;  
    }  
}
```





Analyse InsertionSort

- Aufwand:
 - Anzahl der Vertauschungen / Verschiebungen
 - Anzahl der Vergleiche
 - Vergleiche dominieren Vertauschungen, d.h. es werden (wesentlich) mehr Vergleiche als Vertauschungen benötigt
- Außerdem Unterscheidung:
 - Bester Fall: Liste ist schon sortiert
 - Mittlerer (zu erwartender) Fall: Liste ist unsortiert
 - Schlechtester Fall: z.B. Liste ist absteigend sortiert



Analyse InsertionSort

- Äußere Schleife
 - $n-1$ Durchläufe
- Innere Schleife
 - Bester Fall: schon sortiert
 - 1 Vergleich
 - 0 Verschiebungen
 - Schlechtester Fall: umgekehrt
 - $i-1$ Vergleiche
 - $i-1$ Verschiebungen
 - Durchschnitt
 - $\approx i/2$ Vergleiche
 - $\approx i/2$ Verschiebungen

```
algorithm InsertionSort (F)
Eingabe/Ausgabe: Folge F der Länge n
for i := 2 to n do
    m := F[i];
    j := i;
    while j > 1 do
        if F[j-1] > m then
            F[j] := F[j-1];
            j := j - 1;
        else
            Verlasse innere Schleife
        fi
    od;
    F[j] := m
od;
```



Analyse InsertionSort

- Äußere Schleife
 - n-1 Durchläufe
- Innere Schleife
 - Bester Fall: schon sortiert
 - 1 Vergleich
 - 0 Verschiebungen
 - Schlechtester Fall: umgekehrt
 - i-1 Vergleiche
 - i-1 Verschiebungen
 - Durchschnitt
 - $\approx i/2$ Vergleiche
 - $\approx i/2$ Verschiebungen

$$\sum_{i=2}^n 1 = \sum_{i=1}^{n-1} 1 = n - 1 \approx n$$

$$\sum_{i=2}^n (i - 1) = \sum_{i=1}^{n-1} i = \frac{n(n - 1)}{2} \approx \frac{n^2}{2}$$

$$\sum_{i=2}^n (i/2) \approx \frac{n(n - 1)}{4} \approx \frac{n^2}{4}$$



Analyse 2 InsertionSort

- Starte einen Stapel mit der ersten Karte.
- Wiederhole
 - Nehme die nächste Karte und füge sie an der richtigen Stelle in den neuen Stapel ein.
 - Richtige Stelle finden: Aufwand?
 - Karte einfügen: Aufwand?
 - bis alle Karten einsortiert sind.
- Also: Wenn „verschieben“ nichts kostet, dann
 - Aufwand $\approx n * \log_2 n$



Sortieren durch Selektion

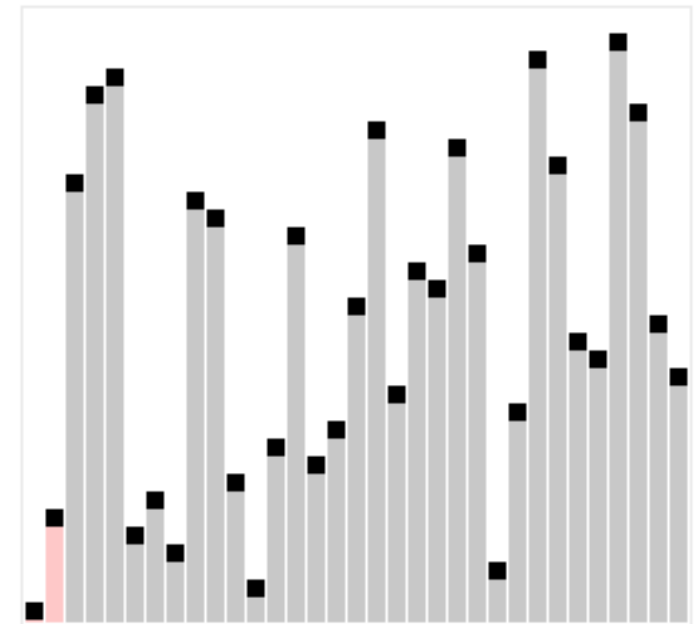
- Idee: Suche jeweils größten Wert, und tausche diesen an die letzte Stelle; fahre dann mit der um 1 kleineren Liste fort.

```
algorithm SelectionSort (F)
  Eingabe: zu sortierende Folge F der Länge n
  p := n;
  while p > 1 do
    g := Index des größten Elements aus F im Bereich 1..p;
    // Teilproblem: findIndexOfMaximum(F,1,p)
    vertausche Werte von F[p] und F[g]
    p := p-1;
  od;
```

Sortieren durch Selektion

- Idee: Suche jeweils größten Wert, und tausche diesen an die letzte Stelle; fahre dann mit der um 1 kleineren Liste fort.

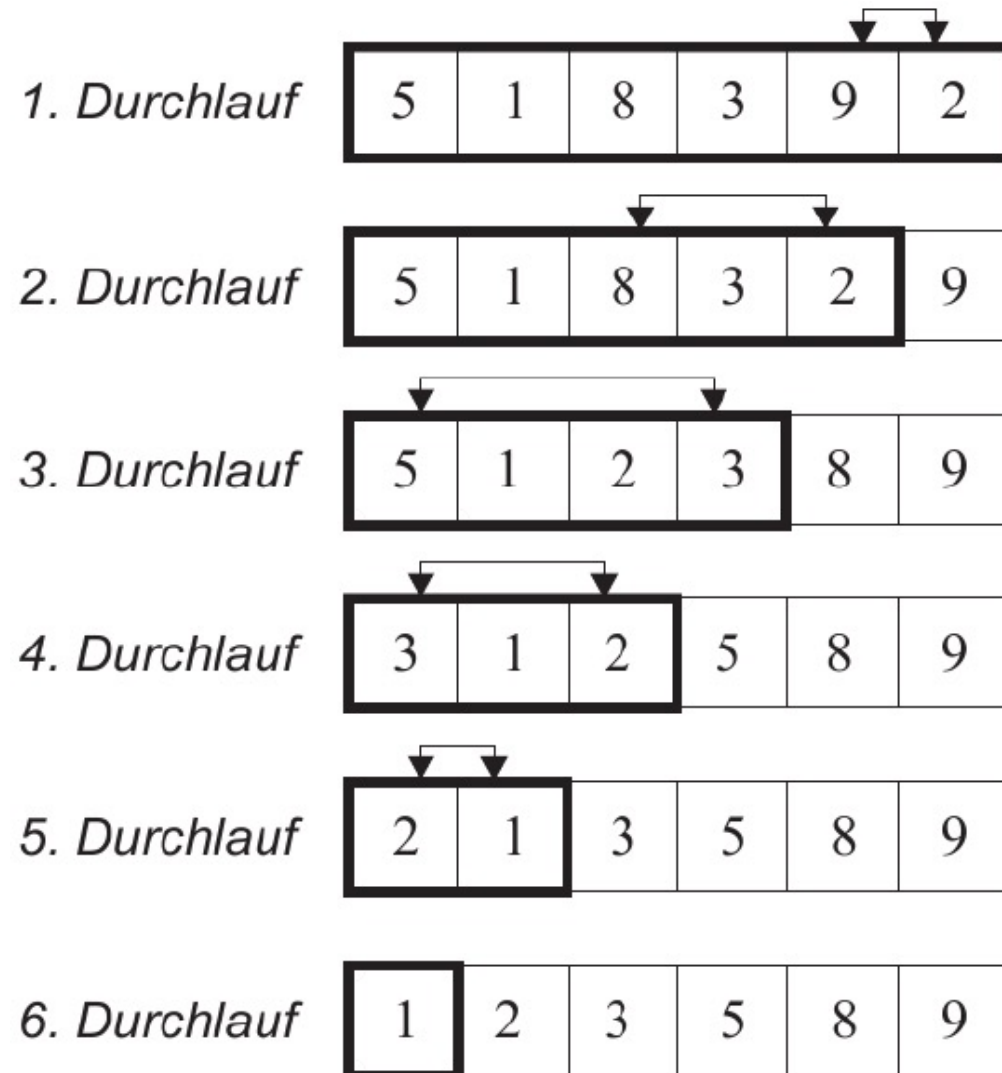
```
algorithm SelectionSort (F)
  Eingabe: zu sortierende Folge F der Länge n
  p := n;
  while p > 1 do
    g := Index des größten Elements aus F im Bereich 1..p;
    // Teilproblem: findIndexOfMaximum(F,1,p)
    vertausche Werte von F[p] und F[g]
    p := p-1;
  od;
```



https://commons.wikimedia.org/wiki/File:Sorting_selection_sort_anim.gif



Beispiel Sortieren durch Selektion / Auswahl





SelectionSort in Java



```
/* Hilfsmethode: Austausch zweier Feldelemente */
static void swap(int[] array, int idx1, int idx2) {
    int tmp = array[idx1];
    array[idx1] = array[idx2];
    array[idx2] = tmp;
}

/* Hilfsmethode für selection sort: findet den Index des größten Elements */
private static int findIndexOfMaximum(int[] array, int start, int stop) {
    int maxPos = start;
    for (int i = start + 1; i <= stop; i++)
        if (array[i] > array[maxPos])
            maxPos = i;
    return maxPos;
}
```



SelectionSort in Java



```
/*  
 * Implementierung des SelectionSort  
 */  
static void selectionSort(int[] array) {  
    int marker = array.length - 1;  
    while (marker >= 0) {  
        int maxPos = findIndexOfMaximum(array, 0, marker);  
        // tausche array[marker] mit diesem Element  
        swap(array, marker, maxPos);  
        marker--;  
    }  
}
```



Analyse SelectionSort

- in jedem Durchlauf das Element von $F[p]$ mit dem größten Element tauschen
- Variable p läuft von $n \dots 1$
 - daher $n-1$ Vertauschungen
- in jedem Durchlauf das größte Element aus $1 \dots p$ ermitteln
 - $p - 1$ Vergleiche
- $(n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1 = \frac{n(n-1)}{2} \approx n^2/2$
- Anzahl Operationen identisch für besten, mittleren und schlechtesten Fall!

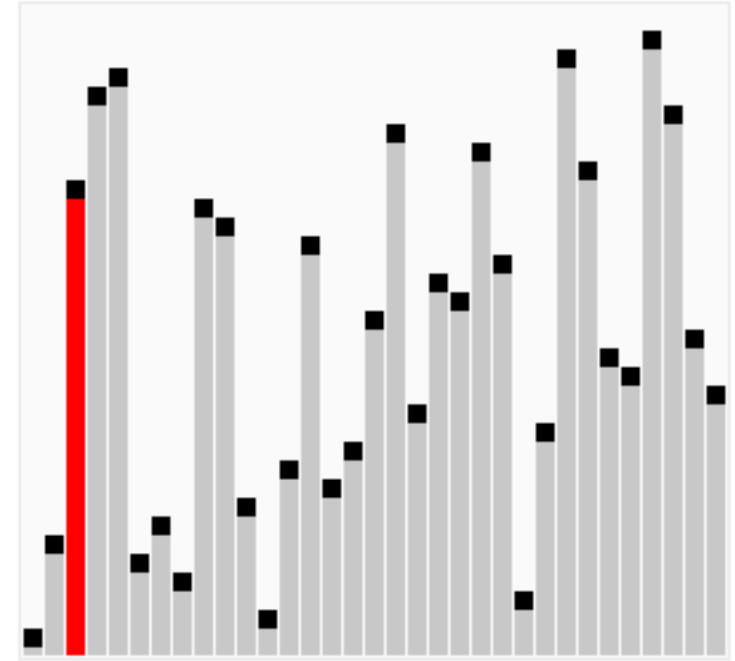


Sortieren durch Vertauschen: BubbleSort

- Luftblasen steigen nach oben, Flüssigkeit nach unten.
- Idee:
 - Teste, ob zwei benachbarte Elemente in der falschen Reihenfolge sind.
 - wenn ja, dann vertausche sie.
 - bis alles sortiert ist.
 - Wenn alle Paare getestet wurden und keines vertauscht!
 - oder: wenn $(n-1)$ -mal das gesamte Feld durchlaufen wurde.

Sortieren durch Vertauschen: BubbleSort

- Luftblasen steigen nach oben, Flüssigkeit nach unten.
- Idee:
 - Teste, ob zwei benachbarte Elemente in der falschen Reihenfolge sind.
 - wenn ja, dann vertausche sie.
 - bis alles sortiert ist.
 - Wenn alle Paare getestet wurden und keines vertauscht!
 - oder: wenn $(n-1)$ -mal das gesamte Feld durchlaufen wurde.



https://commons.wikimedia.org/wiki/Category:Bubble_sort?uselang=de#/media/File:Sorting_bubblesort_anim.gif

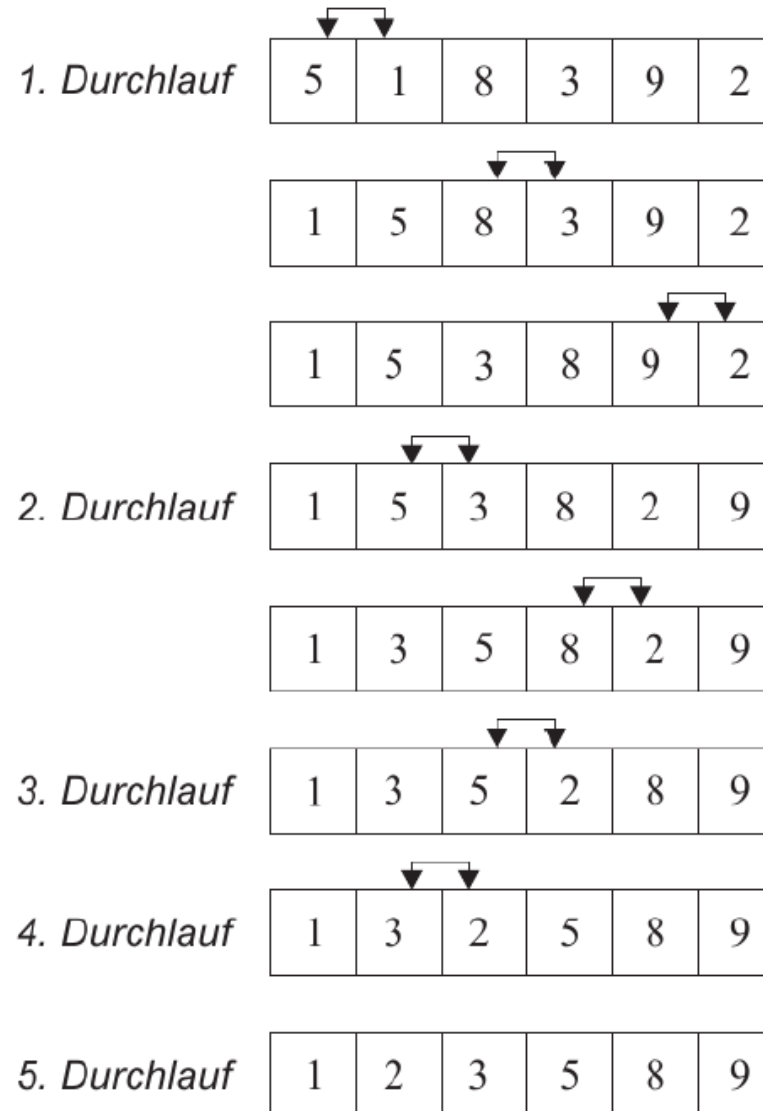


BubbleSort

```
algorithm BubbleSort (F)
Eingabe: zu sortierende Folge F der Länge n
do
  for i := 1 to n-1 do
    if F[i] > F[i+1] then
      vertausche Werte von F[i] und F[i+1]
    fi
  od
until keine Vertauschung mehr aufgetreten
```



Beispiel BubbleSort





BubbleSort

- Optimierungen:
 - Abbrechen, wenn keine Vertauschung nötig war.
 - for-Schleife im j-ten Durchlauf nur bis $n-j$
 - die j größten Elemente sind schon korrekt.
 - Abwechselnd auf- und abwärts „blubbern“
 - in größeren Schritten blubbern → ShellSort



BubbleSort in Java



```
static void bubbleSort(int[] array) {  
    boolean swapped; // Vertauschung?  
    int max = array.length - 1; // obere Feldgrenze  
    do {  
        swapped = false;  
        for (int i = 0; i < max; i++) {  
            if (array[i] > array[i + 1]) {  
                // Elemente vertauschen  
                swap(array, i, i + 1);  
                swapped = true;  
            }  
        }  
        // solange Vertauschung auftritt  
    } while (swapped);  
}
```



BubbleSort in Java



```
static void bubbleSort(int[] array) {  
    boolean swapped; // Vertauschung?  
    int max = array.length - 1; // obere Feldgrenze  
    do {  
        swapped = false;  
        for (int i = 0; i < max; i++) {  
            if (array[i] > array[i + 1]) {  
                // Elemente vertauschen  
                swap(array, i, i + 1);  
                swapped = true;  
            }  
        }  
        max--; // optimierte Version  
        // solange Vertauschung auftritt  
    } while (swapped);  
}
```

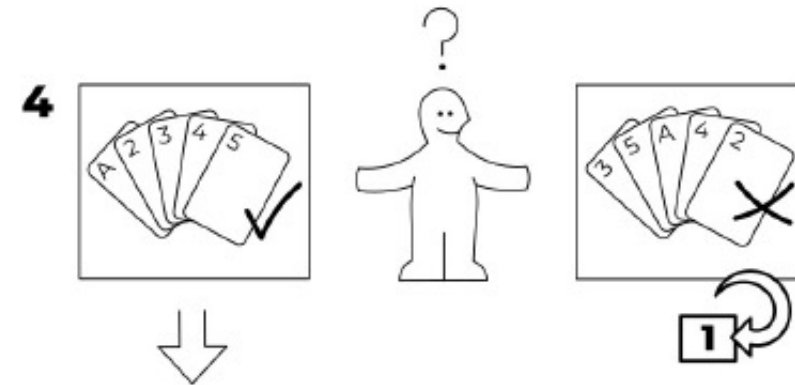
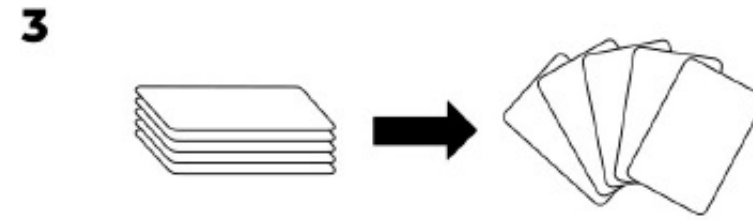
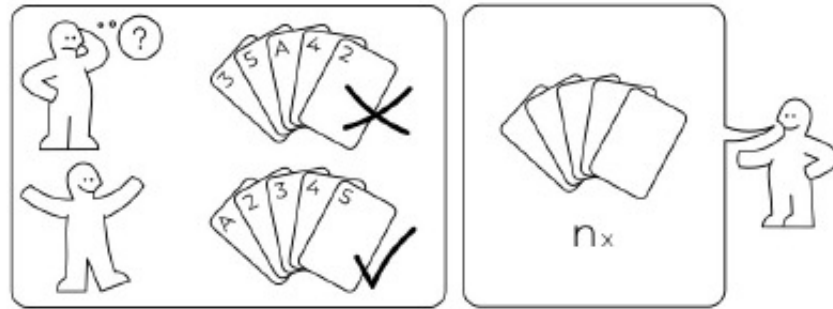


Analyse BubbleSort

- Bester Fall: $n-1$
- Schlechtester Fall
 - $n*(n-1)/2$ (optimiert)
 - $n*(n-1)$
- Durchschnittlicher Fall
 - $n*(n-1)/4$

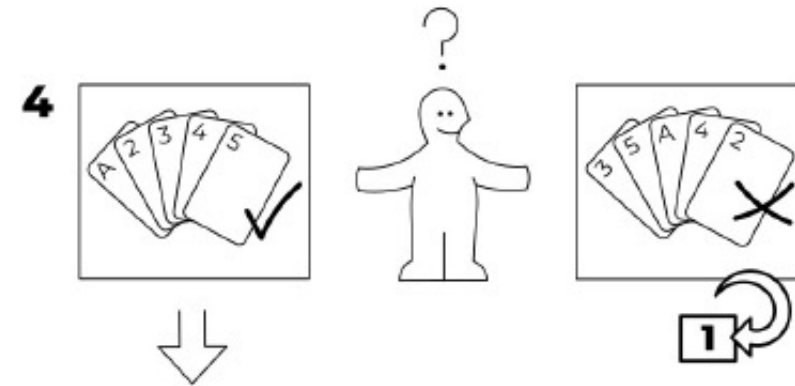
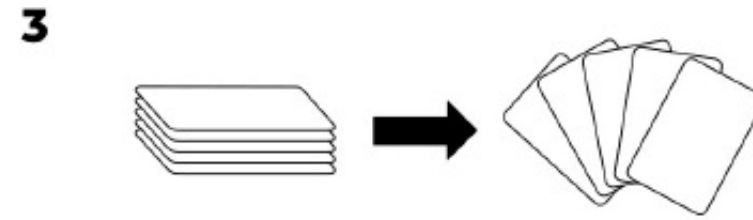
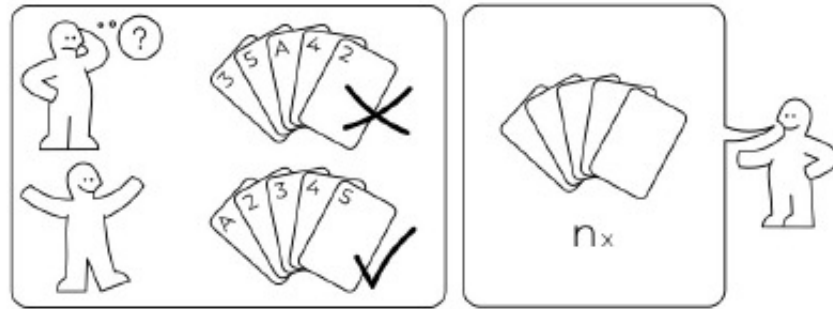
BOGO SÖRT

idea-instructions.com/bogo-sort/
v1.1, CC by-nc-sa 4.0



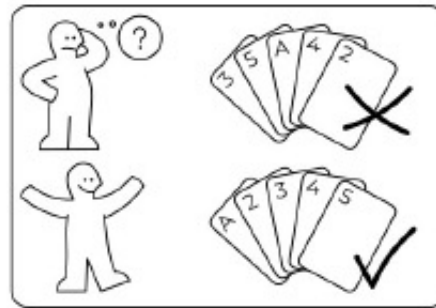
BOGO SÖRT

idea-instructions.com/bogo-sort/
v1.1, CC by-nc-sa 4.0

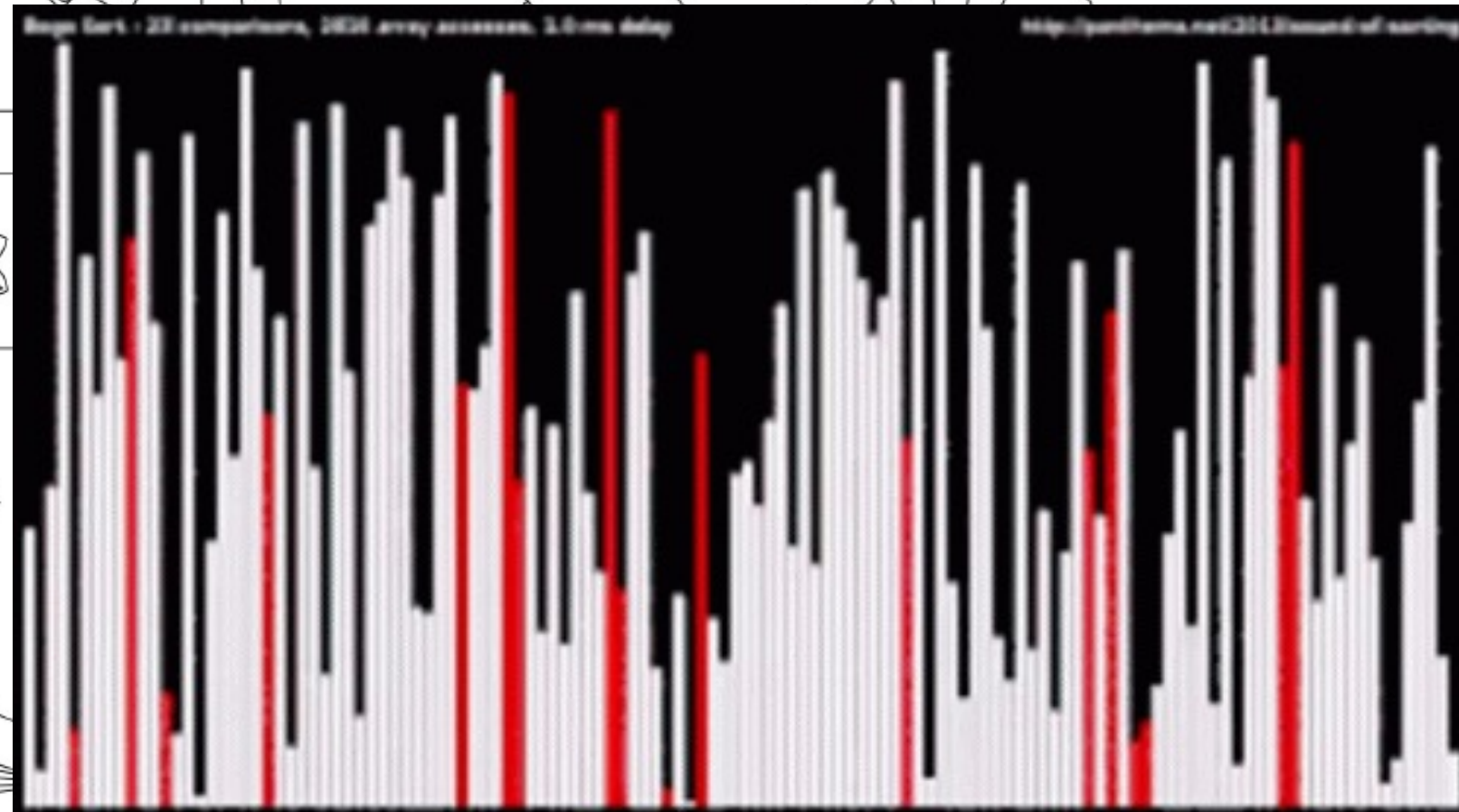
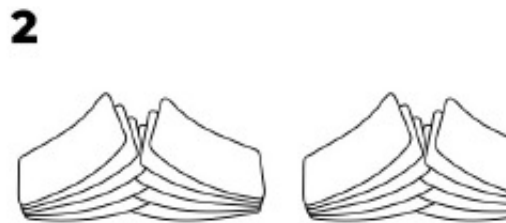


BOGO SÖRT

idea-instructions.com/bogo-sort/
v1.1, CC by-nc-sa 4.0



3



Externe Sortierverfahren

- Bisherige Verfahren: intern
 - alle Daten sind im Speicher
- Externe Verfahren
 - Ursprünglich aus der Magnetband-Zeit:
 - Unsortierte Daten auf Laufwerk 1
 - Sortierte Daten auf Laufwerk 2
 - Hilfsspeicher auf Laufwerk 3



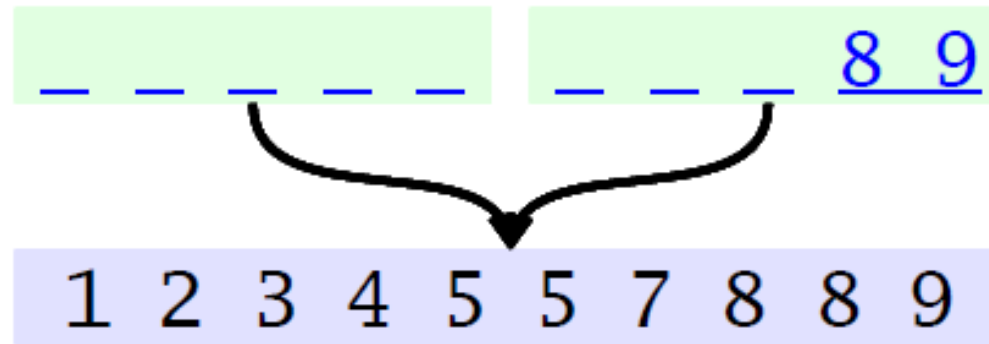


Externe Sortierverfahren

- Datei in Teile teilen, die jeweils intern sortiert werden können
 - Teile müssen einzeln in den Speicher passen
 - bereits sortierte Teile auf Magnetband speichern
- Teile zu sortierter Datei zusammenfügen
 - „merge“
 - drei Laufwerke zur Verfügung:
 - je zwei Teile zusammenmischen
 - zwei Laufwerke lesen
 - auf das dritte Laufwerk schreiben
 - immer das nächstgrößere Element von den beiden Leselaufwerken nehmen.



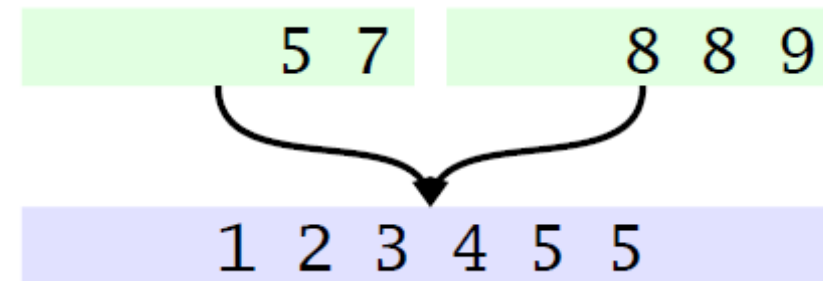
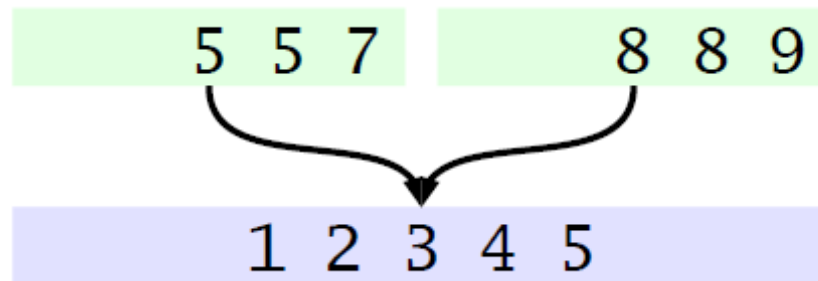
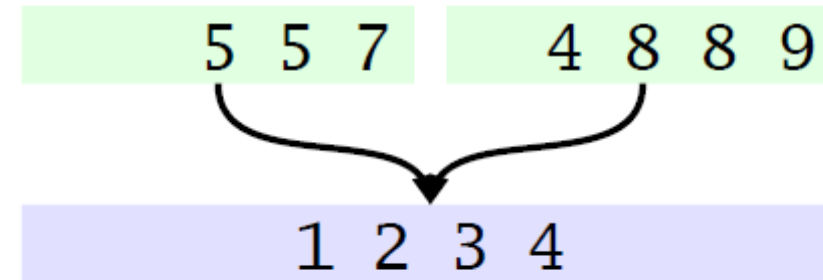
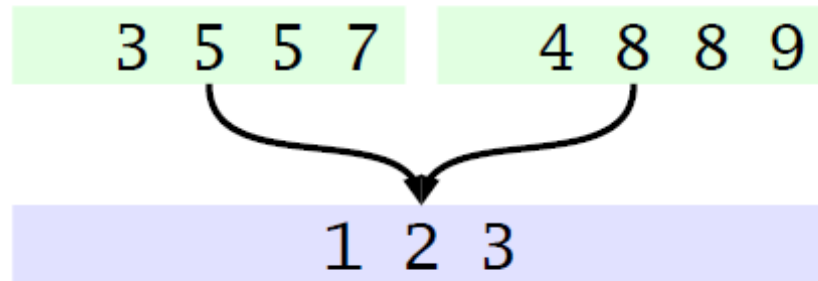
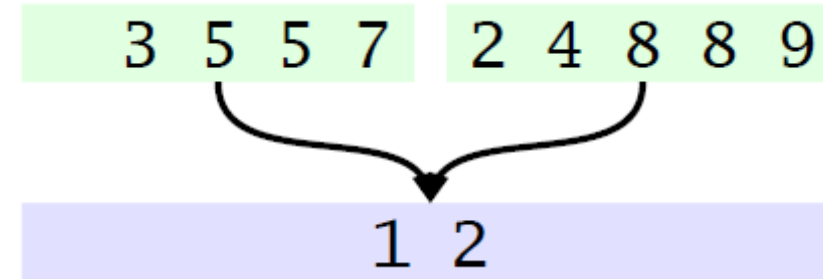
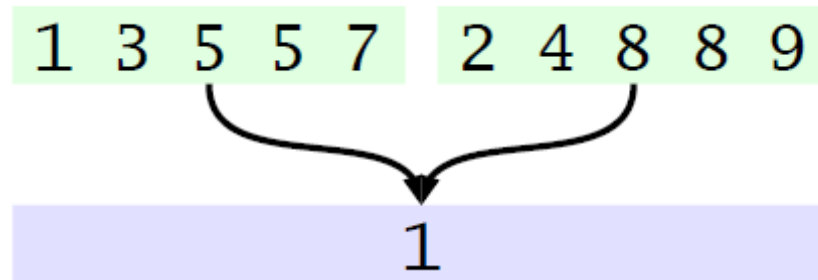
Merge



- Wenn die Teilsequenzen sortiert waren, ist das Resultat sortiert!

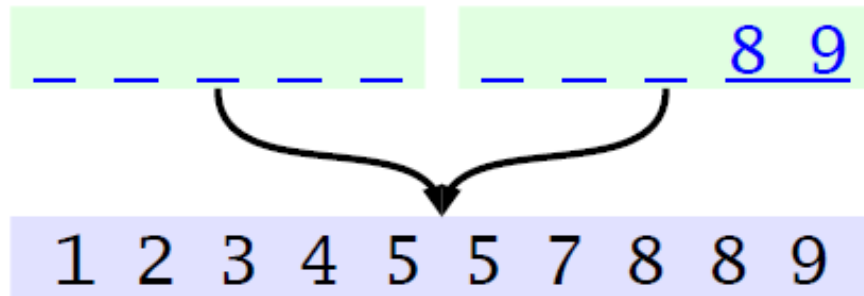
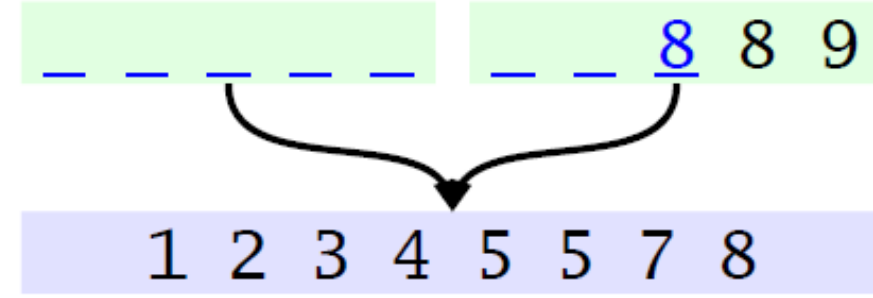
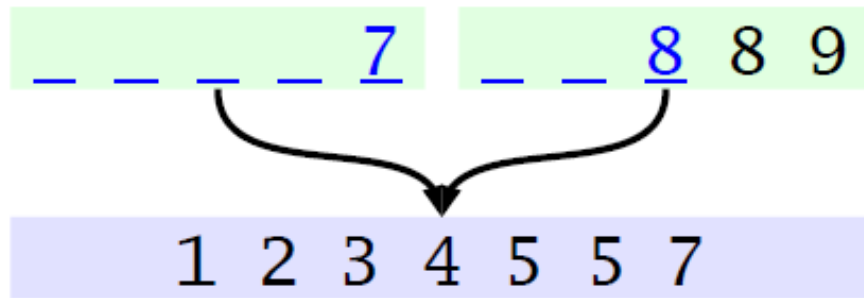


Merge



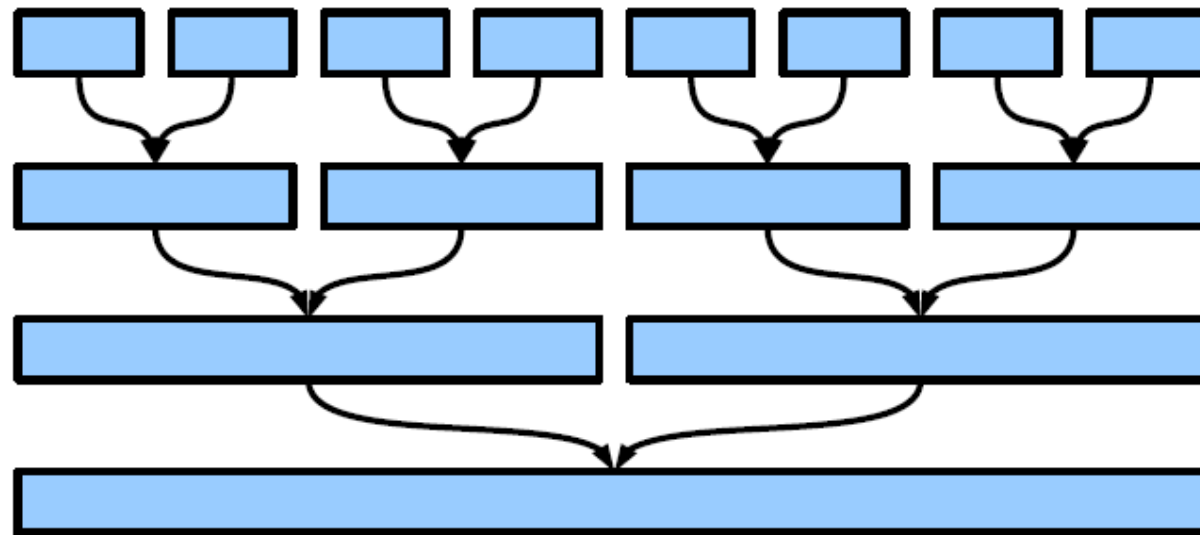


Merge



MergeSort

- Mehr als zwei Teile:
 - jeweils zwei Teile zusammenmischen, Ergebnisse weiter kombinieren!
 - Was passiert, wenn wir mit Teilen der Größe 1 starten?



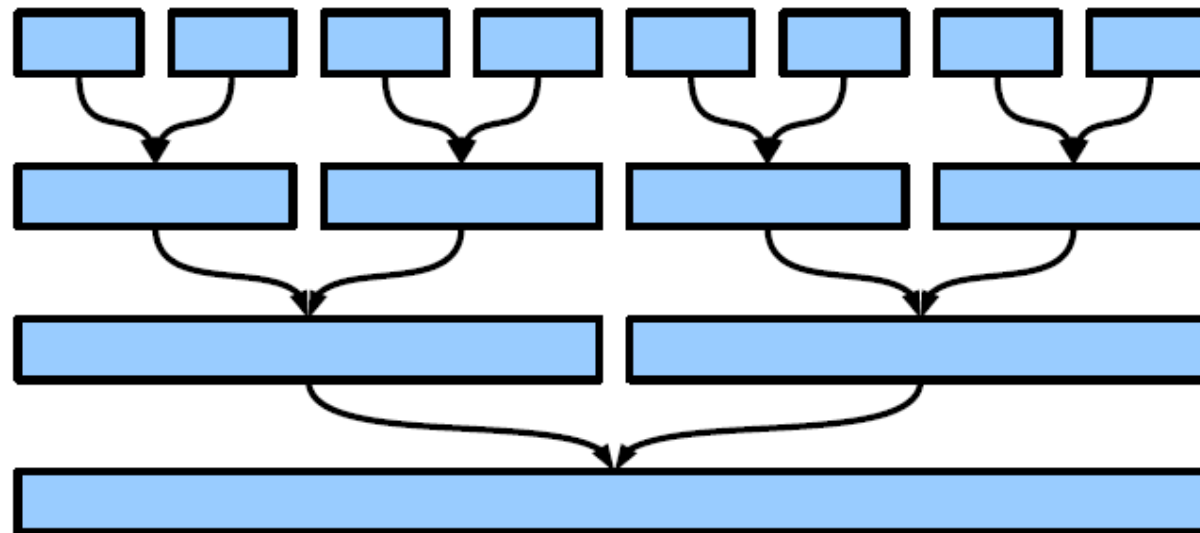


MergeSort



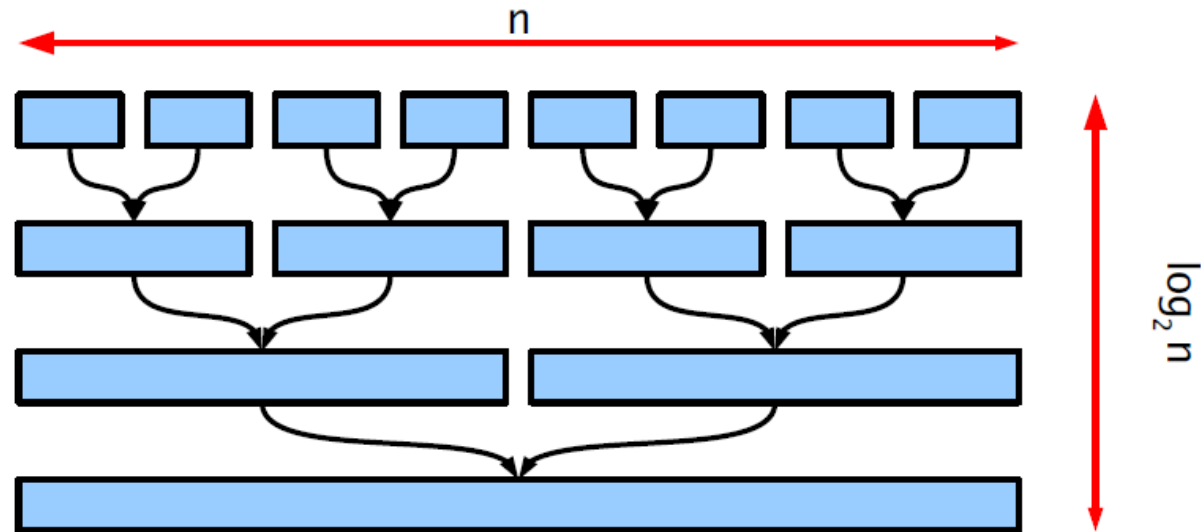
<https://github.com/LucasPilla/Sorting-Algorithms-Visualizer/issues/45#issuecomment-771277015>

- Mehr als zwei Teile:
 - jeweils zwei Teile zusammenmischen, Ergebnisse weiter kombinieren!
 - Was passiert, wenn wir mit Teilen der Größe 1 starten?



MergeSort

- Zerlegen: eigentlich trivial, aber umkopieren!
- Phasen (wie viele „Ebenen“): $\log_2 n$
- jeweils alle n Elemente ansehen (und kopieren) $\Rightarrow n * \log_2 n$





MergeSort

algorithm MergeSort (F) → FS

Eingabe: eine zu sortierende Folge F

Ausgabe: eine sortierte Folge FS

if F einelementig then

 return F

else

 teile F in F1 und F2;

 F1 := MergeSort(F1);

 F2 := MergeSort(F2);

 return Merge(F1, F2)

procedure Merge (F1 , F2) → F

Eingabe: zwei zu sortierende Folgen F1 , F2

Ausgabe: eine sortierte Folge F

F := leere Folge;

while F1 und F2 nicht leer do

 Entferne das kleinere der Anfangselemente aus F1 bzw. F2;

 Füge dieses Element an F an

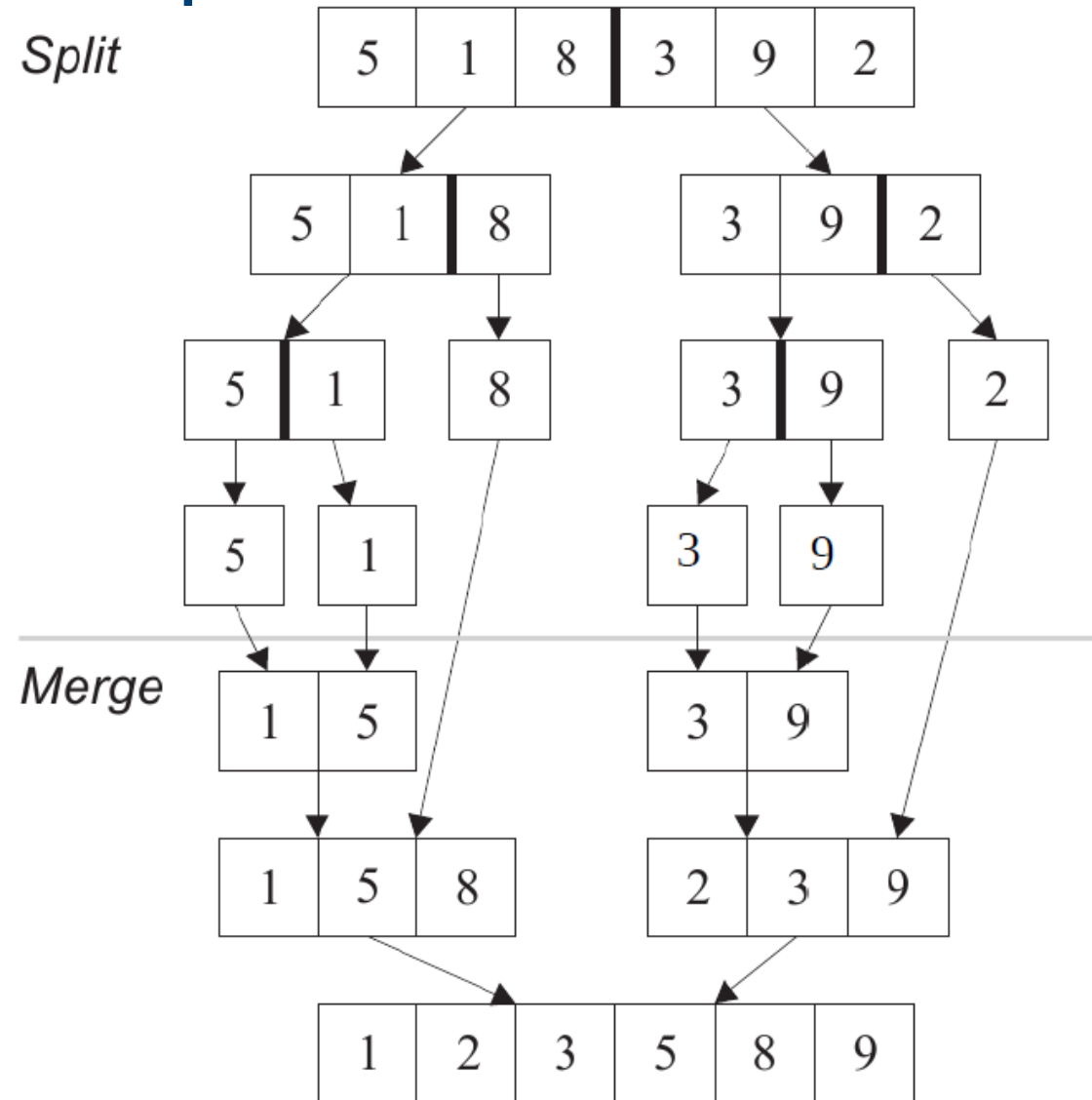
od;

Füge die verbliebene nichtleere Folge F1 oder F2 an F an;

return F



MergeSort: Beispiel





MergeSort in Java

Selbst programmieren!





MergeSort: Analyse

- Anzahl Vergleiche: $n * \log_2 n$
- Speicherplatz: $2*n$
- Es gibt auch 3-Wege Mischen, 4-Wege Mischen
 - Externes Sortiervverfahren mit Festplatten:
 - lade k Teile von der Platte in den Speicher,
 - mische sie mit k-Wege Mischen
- Praxis:
 - für kleine n ist ein n^2 -Verfahren schneller
 - benötigt keinen extra Platz, keine rekursiven Aufrufe
 - Vorteile kombinieren:
 - Rekursion nur bis nlimit
 - Kleinere Teile mit InsertionSort

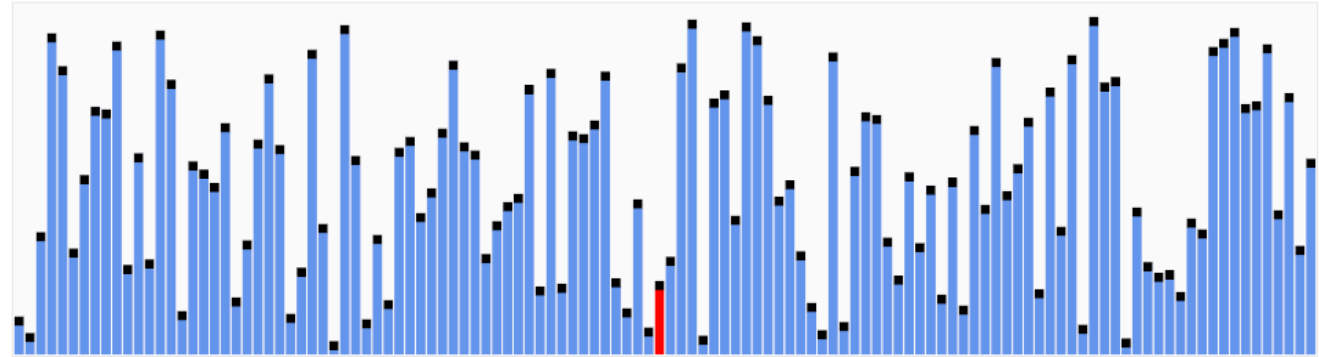


QuickSort

- Idee:
 - rekursive Aufteilung ähnlich wie MergeSort
 - Aufteilung der Teillisten in zwei Hälften bezüglich eines Pivot-Elementes, wobei
 - in einer Liste alle Elemente größer als das Referenzelement sind
 - in der anderen Liste alle Elemente kleiner sind
 - Dadurch wird der Speicherplatz für das Hilfsfeld (MergeSort) nicht benötigt.



QuickSort



<https://commons.wikimedia.org/wiki/File:Quicksort.gif>

- Idee:
 - rekursive Aufteilung ähnlich wie MergeSort
 - Aufteilung der Teillisten in zwei Hälften bezüglich eines Pivot-Elementes, wobei
 - in einer Liste alle Elemente größer als das Referenzelement sind
 - in der anderen Liste alle Elemente kleiner sind
 - Dadurch wird der Speicherplatz für das Hilfsfeld (MergeSort) nicht benötigt.



QuickSort

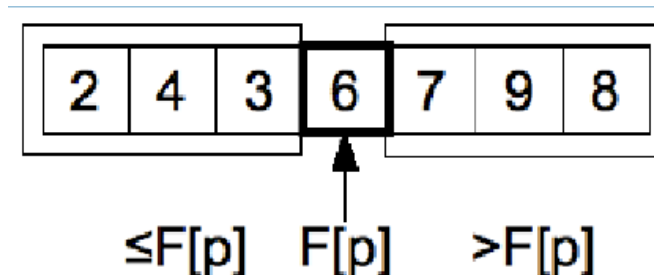
```
algorithm QuickSort (F , u , o)
Eingabe: eine zu sortierende Folge F
         die untere und obere Grenze u, o
if o > u then
    Bestimme Position p des Pivot-Elements;
    pn := Zerlege(F, u, o, p); // pn: neue Position
    QuickSort(F, u, pn - 1); // Rekursion
    QuickSort(F, pn + 1, o); // Rekursion
fi
```

- Wahl des Pivot-Elementes:
 - immer das erste (unsortierte)
 - immer das letzte
 - immer das mittlere
- Wunsch: der Median (Problem?)



QuickSort: Zerlegen durch Vertauschen

- gegeben ein Pivot-Element p
 - durchsuche die Folge von links, bis ein Element $a \geq p$ gefunden ist
 - (das muss nach rechts)!
 - durchsuche die Folge von rechts, bis ein Element $b \leq p$ gefunden ist
 - (das muss nach links)!
 - Vertausche a und b .
- Wiederhole, bis die beiden Zeiger sich treffen.





QuickSort

algorithm **qsort**(F, li, re)

Eingabe Feld F, int li, re

Ausgabe: Feld F wird sortiert.

```
if li < re then
  int pivot = F[(li + re) / 2];
  int i = li;
  int j = re;
  while i < j do // Wenn Indexe nicht gekreuzt
    while F[i] < pivot do // Finde erstes Element größer gleich pivot
      i++;
    od
    while F[j] > pivot do // Finde letztes Element kleiner gleich pivot
      j--;
    od
    swap(F, i, j); // Inhalte vertauschen
  od
  qsort(F, li, j - 1); // linke Partition rekursiv sortieren
  qsort(F, j + 1, re); // rechte Partition rekursiv sortieren
fi
```

```
quickSort(F) {
  qsort(F, 1, F.length);
}
```




QuickSort Beispiel

8 2 1 5 9 7 3
8 2 1 5 9 7 3
3 2 1 5 9 7 8

3 2 1 | 5 | 9 7 8
3 2 1 | 5 | 9 7 8
1 2 3 | 5 | 9 7 8

1 2 3 | 5 | 9 7 8
1 2 3 | 5 | 9 7 8
1 2 3 | 5 | 7 | 9 8

1 2 3 | 5 | 7 | 9 8
1 2 3 | 5 | 7 | 9 8
1 2 3 5 7 8 9

Sortieren auch parallel möglich!



Übung QuickSort

- Führen Sie das Quicksort-Verfahren am Beispiel 9 3 7 1 5 4 8 3 2 durch.
- Notieren Sie jeweils den Zustand des Feldes und wichtige Variablen für jeden (größeren) Schritt.



i:	1	2	3	4	5	6	7	8	9
F[i]:	9	3	7	1	5	4	8	3	2



Analyse Quicksort

- Aufwand analog zu MergeSort abschätzbar
- Bester Fall: $n \cdot \log n$
- Durchschnittlicher Fall: $n \cdot \log n$
- Schlechtester Fall: n^2
 - Wenn das Pivot-Element immer das kleinste oder größte ist.
 - sehr selten!
- aber: im Gegensatz zur MergeSort ist QuickSort durch Vorgehensweise bei Vertauschungen **instabil**

ShellSort

- Modifikation von Sortieren durch Einfügen.
- InsertionSort:
 - Problem war: im schlimmsten Fall wandert ein Element in jedem Durchlauf eine Position weiter.
 - Alle anderen müssen rücken.
- ShellSort:
 - Schiebe erst in großen Sprüngen, dann kleiner.



<https://commons.wikimedia.org/w/index.php?curid=38531293>

D: 5, 3, 1,



Erinnerung InsertionSort

- Äußere Schleife
 - $n-1$ Durchläufe
- Innere Schleife
 - Bester Fall: schon sortiert
 - 1 Vergleich
 - 0 Verschiebungen
 - Schlechtester Fall: umgekehrt
 - $i-1$ Vergleiche
 - $i-1$ Verschiebungen
 - Durchschnitt
 - $\approx i/2$ Vergleiche
 - $\approx i/2$ Verschiebungen

```
algorithm InsertionSort (F)
Eingabe/Ausgabe: Folge F der Länge n
for i := 2 to n do
    m := F[i];
    j := i;
    while j > 1 do
        if F[j-1] > m then
            F[j] := F[j-1];
            j := j - 1;
        else
            Verlasse innere Schleife
        fi
    od;
    F[j] := m
od;
```



Innerer Teil ShellSort

- Schrittweite h

```
for i:= h to n do
  m:= F[i];
  j:= i;
  while j > h && F[j-h] > m do
    F[j] := F[j-h];
    j := j - h;
  od;
  F[j] := m
od;
```

- Schrittweite 1

```
for i:= 2 to n do
  m:= F[i];
  j:= i;
  while j > 1 && F[j-1] > m do
    F[j] := F[j-1];
    j := j - 1;
  od;
  F[j] := m
od;
```



ShellSort

```
int[] spalten = {2147483647, 1131376761, 410151271, 157840433, 58548857,  
                21521774, 8810089, 3501671, 1355339, 543749, 213331,  
                84801, 27901, 11969, 4711, 1968, 815, 271, 111,  
                41, 13, 4, 1};  
for k := 0 to spalten.length do  
  h := spalten[k];  
  for i:= h to n do  
    m:= F[i];  
    j:= i;  
    while j > h && F[j-h] > m do  
      F[j] := F[j-h];  
      j := j - h;  
    od;  
    F[j] := m  
  od;  
od;
```




Sortiervverfahren im Vergleich

Verfahren	Stabilität	Durchschnitt
SelectionSort	Instabil	$\approx n^2/2$
InsertionSort	Stabil	$\approx n^2/4$
BubbleSort	Stabil	$\approx n^2/2$
MergeSort	Stabil	$\approx n \log_2 n$
QuickSort	Instabil	$\approx n \log_2 n$
ShellSort	Instabil	$\approx n^{4/3}$

- Praxis: optimiertes Quicksort, z.B. `java.util.Arrays.sort`
- für stabiles Sortieren: optimiertes `Mergesort`
 - Sortieren von int: Stabilität egal.
 - Sortieren von Object: Stabilität wichtig.
- Vorschau: HeapSort verwendet Bäume.



Sortieren

- Wie aufwendig ist es zu prüfen, ob Daten sortiert sind?
1. Genauso wie Sortieren
 2. Genauso wie binäre Suchen
 3. $n-1$ Schritte im besten Fall
 4. 1 Schritt im besten, $n-1$ Schritte im schlimmsten Fall
 5. $n(n-1)$ Schritte im Durchschnitt



Sortieren

- Wie aufwändig ist es, bei unsortierten Daten das kleinste Element zu finden?
1. Sortieren, dann das erste nehmen $\rightarrow n \log_2 n$
 2. Binäre Suche, also $\log_2 n$
 3. 1 Schritt
 4. n Schritte



Sortieren

- Wie aufwändig ist es, bei sortierten Daten das kleinste Element zu finden?
1. Sortieren, dann das erste nehmen $\rightarrow n \log_2 n$
 2. Binäre Suche, also $\log_2 n$
 3. 1 Schritt
 4. n Schritte